

C++ Standard Library Issues to be moved in Rapperswil

Doc. no. P1082R0

Date: Revised 2018-05-06 at 19:13:12 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2139^{\(i\)}](#). What is a *user-defined* type?

Section: 20.5.4.2.1 [\[namespace.std\]](#), 22.5 [\[syserr\]](#), 23.10.8.1 [\[allocator.uses.traits\]](#), 23.14.11.1 [\[func.bind.isbind\]](#), 23.14.11.2 [\[func.bind.isplace\]](#), 23.14.15 [\[unord.hash\]](#), 23.15.7.6 [\[meta.trans.other\]](#), 25.3.1 [\[locale\]](#), 25.4.1.4 [\[locale.codecvt\]](#), 31.12.1.4 [\[re.register.incr\]](#)
Status: [Ready](#) **Submitter:** Loïc Joly **Opened:** 2012-03-08 **Last modified:** 2018-03-26

Priority: 4

View all other [issues](#) in [\[namespace.std\]](#).

View all issues with [Ready](#) status.

Discussion:

The expression "user-defined type" is used in several places in the standard, but I'm not sure what it means. More specifically, is a type defined in the standard library a user-defined type?

From my understanding of English, it is not. From most of the uses of this term in the standard, it seem to be considered as user defined. In some places, I'm hesitant, e.g. 20.5.4.2.1 [\[namespace.std\]](#) p1:

A program may add a template specialization for any standard library template to namespace std only if the declaration depends on a user-defined type and the specialization meets the standard library requirements for the original template and is not explicitly prohibited.

Does it mean we are allowed to add in the namespace std a specialization for `std::vector<std::pair<T, U>>`, for instance?

Additional remarks from the reflector discussion: The traditional meaning of user-defined types refers to class types and enum types, but the library actually means here user-defined types that are not (purely) library-provided. Presumably a new term - like *user-provided type* - should be introduced and properly defined.

[2012-10 Portland: Move to Deferred]

The issue is real, in that we never define this term and rely on a "know it when I see it" intuition. However, there is a fear that any attempt to pin down a definition is more likely to introduce bugs than solve them - getting the wording for this precisely correct is likely far more work than we are able to give it.

There is unease at simply closing as NAD, but not real enthusiasm to provide wording either. Move to Deferred as we are not opposed to some motivated individual coming back with full wording to review, but do not want to go out of our way to encourage someone to work on this in preference to other issues.

[2014-02-20 Re-open Deferred issues as Priority 4]

[2015-03-05 Jonathan suggests wording]

I dislike the suggestion to change to "user-provided" type because I already find the difference between user-declared / user-provided confusing for special member functions, so I think it would be better to use a completely different term. The core language uses "user-defined conversion sequence" and "user-defined literal" and similar terms for things which the library provides, so I think we should not refer to "user" at all to distinguish entities defined outside the implementation from things provided by the implementation.

I propose "program-defined type" (and "program-defined specialization"), defined below. The P/R below demonstrates the scope of the changes required, even if this name isn't adopted. I haven't proposed a change for "User-defined facets" in [\[locale\]](#).

[Lenexa 2015-05-06]

RS, HT: The core language uses "user-defined" in a specific way, including library things but excluding core language things, let's use a different term.

MC: Agree.

RS: "which" should be "that", x2

RS: Is `std::vector<MyType>` a "program-defined type"?

MC: I think it should be.

TK: `std::vector<int>` seems to take the same path.

JW: `std::vector<MyType>` isn't program-defined, we don't need it to be, anything that depends on that also depends on `=MyType`.

TK: The type defined by an "explicit template specialization" should be a program-defined type.

RS: An implicit instantiation of a "program-defined partial specialization" should also be a program-defined type.

JY: This definition formatting is horrible and ugly, can we do better?

RS: Checking ISO directives.

RS: Define "program-defined type" and "program-defined specialization" instead, to get rid of the angle brackets.

JW redrafting.

[2017-09-12]

Jonathan revises wording as per Lenexa discussion

Previous resolution [SUPERSEDED]:

This wording is relative to N4296.

1. Add a new sub-clause to 20.3 [\[definitions\]](#):

17.3.? [defns.program.defined]

program-defined

<type> a class type or enumeration type which is not part of the C++ standard library and not defined by the implementation. [Note: Types defined by the implementation include extensions (4.1 [\[intro.compliance\]](#)) and internal types used by the library. — end note]

program-defined

<specialization> an explicit template specialization or partial specialization which is not part of the C++ standard library and not defined by the implementation.

2. Change 20.5.4.2.1 [\[namespace.std\]](#) paragraph 1+2:

-1- The behavior of a C++ program is undefined if it adds declarations or definitions to namespace `std` or to a namespace within namespace `std` unless otherwise specified. A program may add a template specialization for any standard library template to namespace `std` only if the declaration depends on a **userprogram**-defined type and the specialization meets the standard library requirements for the original template and is not explicitly prohibited.

-2- The behavior of a C++ program is undefined if it declares

[...]

A program may explicitly instantiate a template defined in the standard library only if the declaration depends on the name of a **userprogram**-defined type and the instantiation meets the standard library requirements for the original template.

3. Change 22.5 [\[sysem\]](#) paragraph 4:

-4- The `is_error_code_enum` and `is_error_condition_enum` may be specialized for **userprogram**-defined types to indicate that such types are eligible for class `error_code` and class `error_condition` automatic conversions,

respectively.

4. Change 23.10.8.1 [\[allocator.uses.traits\]](#) paragraph 1:

-1- *Remarks:* automatically detects [...]. A program may specialize this template to derive from `true_type` for a `user-program`-defined type `T` that does not have a nested `allocator_type` but nonetheless can be constructed with an allocator where either: [...]

5. Change 23.14.11.1 [\[func.bind.isbind\]](#) paragraph 2:

-2- Instantiations of the `is_bind_expression` template [...]. A program may specialize this template for a `user-program`-defined type `T` to have a `BaseCharacteristic` of `true_type` to indicate that `T` should be treated as a subexpression in a `bind` call.

6. Change 23.14.11.2 [\[func.bind.isplace\]](#) paragraph 2:

-2- Instantiations of the `is_placeholder` template [...]. A program may specialize this template for a `user-program`-defined type `T` to have a `BaseCharacteristic` of `integral_constant<int, N>` with $N > 0$ to indicate that `T` should be treated as a placeholder type.

7. Change 23.14.15 [\[unord.hash\]](#) paragraph 1:

The unordered associative containers defined in 23.5 use specializations of the class template `hash` [...], the instantiation `hash<Key>` shall:

- [...]
- [...]
- [...]
- [...]
- satisfy the requirement that the expression `h(k)`, where `h` is an object of type `hash<Key>` and `k` is an object of type `Key`, shall not throw an exception unless `hash<Key>` is a `user-program`-defined specialization that depends on at least one `user-program`-defined type.

8. Change 23.15.7.5 [\[meta.trans.ptr\]](#) Table 57 (common_type row):

Table 57 — Other transformations

Template	Condition	Comments
...		
<code>template <class... T> struct common_type;</code>		The member typedef <code>type</code> shall be defined or omitted as specified below. [...]. A program may specialize this trait if at least one template parameter in the specialization is a <code>user-program</code> -defined type. [...]
...		

9. Change 25.4.1.4 [\[locale.codecvt\]](#) paragraph 3:

-3- The specializations required in Table 81 (22.3.1.1.1) [...]. Other encodings can be converted by specializing on a `user-program`-defined `stateT` type.[...]

10. Change 31.12.1.4 [\[re.regiter.incr\]](#) paragraph 8:

-8- *[Note:* This means that a compiler may call an implementation-specific search function, in which case a `user-program`-defined specialization of `regex_search` will not be called. — *end note]*

[2018-3-14 Wednesday evening issues processing; move to Ready]

After this lands, we need to audit Annex C to find "user-defined type" Example: [diff.cpp03.containers]/3

Proposed resolution:

This wording is relative to [N4687](#).

1. Add a new sub-clause to 20.3 [\[definitions\]](#):

20.3.? [defns.program.defined.spec]

program-defined specialization

explicit template specialization or partial specialization that is not part of the C++ standard library and not defined by the implementation

20.3.? [defns.program.defined.type]

program-defined type

class type or enumeration type that is not part of the C++ standard library and not defined by the implementation, or an instantiation of a program-defined specialization

[Drafting note: ISO directives say the following Note should be labelled as a "Note to entry" but the C++ WP doesn't follow that rule (yet). — end drafting note]

[Note: Types defined by the implementation include extensions (4.1 [\[intro.compliance\]](#)) and internal types used by the library. — end note]

2. Change 20.5.4.2.1 [\[namespace.std\]](#) paragraph 1+2:

-1- The behavior of a C++ program is undefined if it adds declarations or definitions to namespace std or to a namespace within namespace std unless otherwise specified. A program may add a template specialization for any standard library template to namespace std only if the declaration depends on a **userprogram**-defined type and the specialization meets the standard library requirements for the original template and is not explicitly prohibited.

-2- The behavior of a C++ program is undefined if it declares

[...]

A program may explicitly instantiate a template defined in the standard library only if the declaration depends on the name of a **userprogram**-defined type and the instantiation meets the standard library requirements for the original template.

3. Change 22.5 [\[syserr\]](#) paragraph 4:

-4- The is_error_code_enum and is_error_condition_enum may be specialized for **userprogram**-defined types to indicate that such types are eligible for class error_code and class error_condition automatic conversions, respectively.

4. Change 23.10.8.1 [\[allocator.uses.traits\]](#) paragraph 1:

-1- Remarks: automatically detects [...]. A program may specialize this template to derive from true_type for a **userprogram**-defined type T that does not have a nested allocator_type but nonetheless can be constructed with an allocator where either: [...]

5. Change 23.14.11.1 [\[func.bind.isbind\]](#) paragraph 2:

-2- Instantiations of the is_bind_expression template [...]. A program may specialize this template for a **userprogram**-defined type T to have a BaseCharacteristic of true_type to indicate that T should be treated as a subexpression in a bind call.

6. Change 23.14.11.2 [\[func.bind.isplace\]](#) paragraph 2:

-2- Instantiations of the is_placeholder template [...]. A program may specialize this template for a **userprogram**-defined type T to have a BaseCharacteristic of integral_constant<int, N> with $N > 0$ to indicate that T should be treated as a placeholder type.

7. Change 23.14.15 [\[unord.hash\]](#) paragraph 1:

The unordered associative containers defined in 23.5 use specializations of the class template hash [...], the instantiation hash<Key> shall:

- [...]
- [...]
- [...]
- [...]

- satisfy the requirement that the expression $h(k)$, where h is an object of type `hash<key>` and k is an object of type `Key`, shall not throw an exception unless `hash<key>` is a `userprogram`-defined specialization that depends on at least one `userprogram`-defined type.

8. Change 23.15.7.5 [\[meta.trans.ptr\]](#) Table 57 (common_type row):

Table 57 — Other transformations		
Template	Condition	Comments
...		
<code>template <class... T> struct common_type;</code>		The member typedef <code>type</code> shall be defined or omitted as specified below. [...]. A program may specialize this trait if at least one template parameter in the specialization is a <code>userprogram</code> -defined type. [...]
...		

9. Change 25.4.1.4 [\[locale.codecvt\]](#) paragraph 3:

-3- The specializations required in Table 81 (22.3.1.1.1) [...]. Other encodings can be converted by specializing on a `userprogram`-defined `stateT` type.[...]

10. Change 31.12.1.4 [\[re.regiter.incr\]](#) paragraph 8:

-8- [Note: This means that a compiler may call an implementation-specific search function, in which case a `userprogram`-defined specialization of `regex_search` will not be called. — end note]

2970⁽ⁱ⁾. Return type of `std::visit` misspecified

Section: 23.7.7 [\[variant.visit\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-05-31 **Last modified:** 2018-03-05

Priority: 2

View other [active issues](#) in `[variant.visit]`.

View all other [issues](#) in `[variant.visit]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

`[variant.visit]`/1 correctly uses "type and value category", but then p3 describes the return type of `visit` to be "the common type of all possible *INVOKE* expressions of the *Effects*: element." The type of an expression is never a reference type, due to `[expr]/5` removing the referenceness "prior to any further analysis", so this wording as written says that `visit` always returns a non-reference type, which is presumably not the intent.

[2017-07 Toronto Monday issue prioritization]

Priority 2; Matt to provide wording

[2018-01-11, Thomas Köppe comments and suggests wording]

The return type of `std::visit` (originating by [P0088R3](#) accepted during the Oulo 2016 meeting) is currently misspecified and refers only to the common *type* of all the possible visitation calls, without attention to the value category. This seems unintended, and we should preserve the value category.

[2017-01-24, Daniel comments]

This issue should be reviewed in common with LWG [3052](#).

[2018-02-23 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4727](#).

1. Modify 23.7.7 [\[variant.visit\]](#) as indicated:

```
template<class Visitor, class... Variants>
constexpr see below visit(Visitor&& vis, Variants&&... vars);
```

[...]

-3- Returns: $e(m)$, where m is the pack for which m_i is $\text{vars}_i.\text{index}()$ for all $0 \leq i < n$. The return type is the type of $e(m)$. ~~decltype($e(m)$).~~

[...]

[3058\(i\)](#). Parallel adjacent_difference shouldn't require creating temporaries

Section: 29.8.11 [\[adjacent.difference\]](#) Status: [Ready](#) Submitter: Billy O'Neal III Opened: 2018-02-02 Last modified: 2018-03-26

Priority: 3

View all other [issues](#) in [\[adjacent.difference\]](#).

View all issues with [Ready](#) status.

Discussion:

Parallel adjacent_difference is presently specified to "create a temporary object whose type is ForwardIterator1's value type". Serial adjacent_difference does that because it needs to work with input iterators, and needs to work when the destination range exactly overlaps the input range. The parallel version requires forward iterators and doesn't allow overlap, so it can avoid making these temporaries.

[2018-02-13, Priority set to 3 after mailing list discussion]

[2018-3-14 Wednesday evening issues processing; remove 'const' before minus and move to Ready.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4713](#).

1. Modify 29.8.11 [\[adjacent.difference\]](#) as indicated:

```
template<class InputIterator, class OutputIterator>
OutputIterator
adjacent_difference(InputIterator first, InputIterator last, OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
adjacent_difference(ExecutionPolicy&& exec,
                    ForwardIterator1 first, ForwardIterator1 last, ForwardIterator2 result);

template<class InputIterator, class OutputIterator, class BinaryOperation>
OutputIterator
adjacent_difference(InputIterator first, InputIterator last,
                    OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
class BinaryOperation>
ForwardIterator2
adjacent_difference(ExecutionPolicy&& exec,
                    ForwardIterator1 first, ForwardIterator1 last,
                    ForwardIterator2 result, BinaryOperation binary_op);
```

~~Let T be the value type of $\text{decltype}(\text{first})$. For the overloads that do not take an argument binary_op , let binary_op be an lvalue that denotes an object of type $\text{const minus}<>$.~~

-1- Requires:

(1.1) — For the overloads with no ExecutionPolicy, ~~InputIterator's value type T~~ shall be MoveAssignable (Table 25) and shall be constructible from the type of $*\text{first}$. acc (defined below) shall be writable (27.2.1 [\[iterator.requirements.general\]](#)) to the result output iterator. The result of the expression ~~$\text{val} = \text{std::move}(\text{acc})$ or $\text{binary_op}(\text{val}, \text{std::move}(\text{acc}))$~~ shall be writable to the result output iterator.

(1.2) — For the overloads with an ExecutionPolicy, the ~~value type of ForwardIterator1 shall be CopyConstructible (Table 24), constructible from the expression *first - *first or binary_op(*first, *first), and assignable to the value type of ForwardIterator2~~ result of the expressions `binary_op(*first, *first)` and `*first` shall be writable to result.

(1.3) — [...]

-2- *Effects*: For the overloads with no ExecutionPolicy and a non-empty range, the function creates an accumulator `acc` whose type is ~~InputIterator's value type~~ of type `T`, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1, last)` in order, creates an object `val` whose type is ~~InputIterator's value type~~ `T`, initializes it with `*i`, computes ~~`val - std::move(acc)` or `binary_op(val, std::move(acc))`~~, assigns the result to `*(result + (i - first))`, and move assigns from `val` to `acc`.

-3- For the overloads with an ExecutionPolicy and a non-empty range, ~~first the function creates an object whose type is ForwardIterator1's value type, initializes it with *first, and assigns the result to *result. Then for every d in [1, last - first - 1], creates an object val whose type is ForwardIterator1's value type, initializes it with *(first + d) - *(first + d - 1) or binary_op(*(first + d), *(first + d - 1)), and assigns the result to *(result + d) performs *result = *first. Then, for every d in [1, last - first - 1], performs *(result + d) = binary_op(*(first + d), *(first + (d - 1))).~~

Proposed resolution:

This wording is relative to [N4713](#).

1. Modify 29.8.11 [\[adjacent.difference\]](#) as indicated:

```
template<class InputIterator, class OutputIterator>
OutputIterator
adjacent_difference(InputIterator first, InputIterator last, OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
adjacent_difference(ExecutionPolicy&& exec,
                    ForwardIterator1 first, ForwardIterator1 last, ForwardIterator2 result);

template<class InputIterator, class OutputIterator, class BinaryOperation>
OutputIterator
adjacent_difference(InputIterator first, InputIterator last,
                    OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
ForwardIterator2
adjacent_difference(ExecutionPolicy&& exec,
                    ForwardIterator1 first, ForwardIterator1 last,
                    ForwardIterator2 result, BinaryOperation binary_op);
```

~~Let `T` be the value type of `decltype(first)`. For the overloads that do not take an argument `binary_op`, let `binary_op` be an lvalue that denotes an object of type `minus<>`.~~

-1- *Requires*:

(1.1) — For the overloads with no ExecutionPolicy, ~~InputIterator's value type~~ `T` shall be MoveAssignable (Table 25) and shall be constructible from the type of `*first`. `acc` (defined below) shall be writable (27.2.1 [\[iterator.requirements.general\]](#)) to the result output iterator. The result of the expression ~~`val - std::move(acc)` or `binary_op(val, std::move(acc))`~~ shall be writable to the result output iterator.

(1.2) — For the overloads with an ExecutionPolicy, the ~~value type of ForwardIterator1 shall be CopyConstructible (Table 24), constructible from the expression *first - *first or binary_op(*first, *first), and assignable to the value type of ForwardIterator2~~ result of the expressions `binary_op(*first, *first)` and `*first` shall be writable to result.

(1.3) — [...]

-2- *Effects*: For the overloads with no ExecutionPolicy and a non-empty range, the function creates an accumulator `acc` whose type is ~~InputIterator's value type~~ of type `T`, initializes it with `*first`, and assigns

the result to *result. For every iterator i in [first + 1, last) in order, creates an object val whose type is ~~InputIterator's value type~~ **T**, initializes it with *i, computes ~~val = std::move(acc); or~~ **binary_op(val, std::move(acc))**, assigns the result to ***(result + (i - first))**, and move assigns from val to acc.

-3- For the overloads with an ExecutionPolicy and a non-empty range, ~~first the function creates an object whose type is ForwardIterator's value type, initializes it with *first, and assigns the result to *result. Then for every d in [1, last - first - 1], creates an object val whose type is ForwardIterator's value type, initializes it with *(first + d) = *(first + d - 1) or binary_op(*(first + d), *(first + d - 1)), and assigns the result to *(result + d) performs *result = *first. Then, for every d in [1, last - first - 1], performs *(result + d) = binary_op(*(first + d), *(first + (d - 1))).~~

[3062^{\(i\)}](#). Unnecessary decay_t in is_execution_policy_v should be remove_cvref_t

Section: 28.4.5 [\[algorithms.parallel.overloads\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2018-02-07 **Last modified:** 2018-02-20

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Our compiler throughput friends were hissing at us about throughput regressions in C++17 mode caused by the addition of the parallel algorithms' signatures. One change to reduce the throughput impact would be to remove unnecessary decay here, as LWG has done in other places recently.

[2018-02-13 Moved to Tentatively Ready after 7 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Modify 28.4.5 [\[algorithms.parallel.overloads\]](#) as indicated:

-4- Parallel algorithms shall not participate in overload resolution unless **is_execution_policy_v<**~~decay_remove_cvref_t~~**<ExecutionPolicy>>** is true.

[3067^{\(i\)}](#). recursive_directory_iterator::pop must invalidate

Section: 30.11.13.1 [\[fs.rec.dir.itr.members\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2018-02-25 **Last modified:** 2018-03-13

Priority: 0

View other [active issues](#) in [\[fs.rec.dir.itr.members\]](#).

View all other [issues](#) in [\[fs.rec.dir.itr.members\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

recursive_directory_iterator::pop is effectively a "supercharged" operator++: it advances the iterator forward as many steps as are necessary to reach the next entry in the parent directory. Just as is the case for operator++, pop must be allowed to invalidate iterator copies to allow efficient implementation. The most efficient fix seems to be borrowing the invalidation wording from 27.2.3 [\[input.iterators\]](#) Table 87's specification for the required ++r expression for input iterators.

[2018-03-06 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4727](#).

1. Change 30.11.13.1 [\[fs.rec.dir.itr.members\]](#) as indicated:


```
void pop();
void pop(error_code& ec);
```

-26- *Effects*: If `depth() == 0`, set `*this` to `recursive_directory_iterator()`. Otherwise, cease iteration of the directory currently being iterated over, and continue iteration over the parent directory.

-?- *Postconditions*: Any copies of the previous value of `*this` are no longer required either to be dereferenceable or to be in the domain of `==`.

-27- *Throws*: As specified in 30.11.6 [\[fs.err.report\]](#).

[3071^{\(i\)}](#). `[networking.ts]` `read_until` still refers to "input sequence"

Section: 17.9 `[networking.ts::buffer.read.until]` **Status:** [Tentatively Ready](#) **Submitter:** Christopher Kohlhoff **Opened:** 2018-02-26 **Last modified:** 2018-03-13

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: `networking.ts`

When specifying `DynamicBuffers` and their related operations, early drafts of the Networking TS described the buffers in terms of their "input sequence" and "output sequence". This was changed to "readable bytes" and "writable bytes" respectively. Unfortunately, some instances of "input sequence" were missed in section 17.9 `[networking.ts::buffer.read.until]`.

[2018-03-06 Moved to Tentatively Ready after 5 positive votes on `c++std-lib`.]

Proposed resolution:

This wording is relative to [N4711](#).

1. Change 17.9 `[networking.ts::buffer.read.until]` as indicated:

```
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, char delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
                      char delim, error_code& ec);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, string_view delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
                      string_view delim, error_code& ec);
```

-1- *Effects*: Reads data from the buffer-oriented synchronous read stream (17.1.1 `[networking.ts::buffer.stream.reqmts.syncreadstream]`) object stream by performing zero or more calls to the stream's `read_some` member function, until the `input sequence readable bytes` of the dynamic buffer (16.2.4 `[networking.ts::buffer.reqmts.dynamicbuffer]`) object `b` contains the specified delimiter `delim`.

-2- Data is placed into the dynamic buffer object `b`. A mutable buffer sequence (16.2.1) is obtained prior to each `read_some` call using `b.prepare(N)`, where `N` is an unspecified value such that `N <= max_size() - size()`. [Note: Implementations are encouraged to use `b.capacity()` when determining `N`, to minimize the number of `read_some` calls performed on the stream. — end note] After each `read_some` call, the implementation performs `b.commit(n)`, where `n` is the return value from `read_some`.

-3- The synchronous `read_until` operation continues until:

(3.1) — the `input sequence readable bytes` of `b` contains the delimiter `delim`; or

(3.2) — `b.size() == b.max_size()`; or

(3.3) — an asynchronous `read_some` operation fails.

-4- On exit, if the `input sequence readable bytes` of `b` contains the delimiter, `ec` is set such that `!ec` is true. Otherwise, if `b.size() == b.max_size()`, `ec` is set such that `ec == stream_errc::not_found`. If `b.size() < b.max_size()`, `ec` contains the `error_code` from the most recent `read_some` call.

-5- Returns: The number of bytes in the input sequence of readable bytes in b up to and including the delimiter, if present. [Note: On completion, the buffer may contain additional bytes following the delimiter. — end note] Otherwise returns 0.

3074⁽ⁱ⁾. Non-member functions for valarray should only deduce from the valarray

Section: 29.7.3 [\[valarray.nonmembers\]](#) Status: [Tentatively Ready](#) Submitter: Jonathan Wakely Opened: 2018-02-28 Last modified: 2018-03-13

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The expression `(std::valarray<double>{} * 2)` is ill-formed, because argument deduction fails for:

```
template<class T>
valarray<T> operator*(const valarray<T>&, const T&);
```

Is there any reason to try and deduce the argument from the scalar, instead of only deducing from the valarray and allowing implicit conversions to the scalar? i.e.

```
template<class T>
valarray<T> operator*(const valarray<T>&, const typename valarray<T>::value_type&);
```

[2018-03-07 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 29.7.1 [\[valarray.syn\]](#), header `<valarray>` synopsis, as indicated:

```
[...]
template<class T> valarray<T> operator* (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator* (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator* (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator/ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator/ (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator/ (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator% (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator% (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator% (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator+ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator+ (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator+ (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator- (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator- (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator- (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator^ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator^ (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator^ (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator& (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator& (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator& (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator| (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator| (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator| (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator<< (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator<< (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator<< (const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> operator>> (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator>> (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> operator>> (const typename valarray<T>::value_type&, const valarray<T>&);
```



```
[...]
template<class T> valarray<bool> operator==(const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator==(const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator!=(const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator< (const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator> (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator> (const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator<= (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator<= (const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator>= (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator>= (const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator&& (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator&& (const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<bool> operator|| (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator|| (const typename valarray<T>::value_type&, const valarray<T>&);
[...]
```

4. Edit 29.7.3.3 [\[valarray.transcend\]](#) as indicated:

```
template<class T> valarray<T> abs (const valarray<T>&);
template<class T> valarray<T> acos (const valarray<T>&);
template<class T> valarray<T> asin (const valarray<T>&);
template<class T> valarray<T> atan (const valarray<T>&);
template<class T> valarray<T> atan2(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> atan2(const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> atan2(const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<T> cos (const valarray<T>&);
template<class T> valarray<T> cosh (const valarray<T>&);
template<class T> valarray<T> exp (const valarray<T>&);
template<class T> valarray<T> log (const valarray<T>&);
template<class T> valarray<T> log10(const valarray<T>&);
template<class T> valarray<T> pow (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> pow (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> pow (const typename valarray<T>::value_type&, const valarray<T>&);
[...]
```

3076⁽ⁱ⁾. basic_string CTAD ambiguity

Section: 24.3.2.2 [\[string.cons\]](#) **Status:** [Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2018-03-03 **Last modified:** 2018-03-26

Priority: Not Prioritized

View all other [issues](#) in [\[string.cons\]](#).

View all issues with [Ready](#) status.

Discussion:

The following code fails to compile for surprising reasons.

```
#include <string>
#include <string_view>

using namespace std;

int main()
{
    string s0;
    basic_string s1(s0, 1, 1);
    // WANT: basic_string(const basic_string&, size_type, size_type, const Allocator& = Allocator())
    // CONFLICT: basic_string(size_type, charT, const Allocator&)

    basic_string s2("cat"sv, 1, 1);
    // WANT: basic_string(const T&, size_type, size_type, const Allocator& = Allocator())
    // CONFLICT: basic_string(size_type, charT, const Allocator&)

    basic_string s3("cat", 1);
    // WANT: basic_string(const charT *, size_type, const Allocator& = Allocator())
    // CONFLICT: basic_string(const charT *, const Allocator&)
}
```

For s1 and s2, the signature `basic_string(size_type, charT, const Allocator&)` participates in CTAD. `size_type` is non-deduced (it will be substituted later, so the compiler can't immediately realize that `s0` or `"cat"sv` are totally non-viable arguments). `charT` is deduced to

be `int` (weird, but not the problem). Finally, `Allocator` is deduced to be `int`. Then the compiler tries to substitute for `size_type`, but this ends up giving `int` to `allocator_traits` in a non-SFINAE context, so compilation fails.

`s3` fails for a slightly different reason. `basic_string(const charT *, const Allocator&)` participates in CTAD, deducing `charT` to be `char` (good) and `Allocator` to be `int`. This is an exact match, which is better than the constructor that the user actually wants (where `int` would need to be converted to `size_type`, which is unsigned). So CTAD deduces `basic_string<char, char_traits<char>, int>`, which is the wrong type.

This problem appears to be unique to `basic_string` and its heavily overloaded set of constructors. I haven't figured out how to fix it by adding (non-greedy) deduction guides. The conflicting constructors are always considered during CTAD, regardless of whether deduction guides are provided that correspond to the desired or conflicting constructors. (That's because deduction guides are preferred as a late tiebreaker in overload resolution; if a constructor provides a better match it will be chosen before tiebreaking.) It appears that we need to constrain the conflicting constructors themselves; this will have no effect on actual usage (where `Allocator` will be an allocator) but will prevent CTAD from considering them for non-allocators. As this is unusual, I believe it deserves a *Note*. This has been implemented in MSVC.

[2018-3-14 Wednesday evening issues processing; move to Ready]

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 24.3.2.2 [\[string.cons\]](#) as indicated:

```
basic_string(const charT* s, const Allocator& a = Allocator());
```

-14- *Requires*: `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

-15- *Effects*: Constructs an object of class `basic_string` and determines its initial string value from the array of `charT` of length `traits::length(s)` whose first element is designated by `s`.

-16- *Postconditions*: `data()` points at the first element of an allocated copy of the array whose first element is pointed at by `s`, `size()` is equal to `traits::length(s)`, and `capacity()` is a value at least as large as `size()`.

[-?- Remarks: Shall not participate in overload resolution if `Allocator` is a type that does not qualify as an allocator (26.2.1 [\[container.requirements.general\]](#)). [Note: This affects class template argument deduction. — end note]

```
basic_string(size_type n, charT c, const Allocator& a = Allocator());
```

-17- *Requires*: `n < npos`.

-18- *Effects*: Constructs an object of class `basic_string` and determines its initial string value by repeating the char-like object `c` for all `n` elements.

-19- *Postconditions*: `data()` points at the first element of an allocated array of `n` elements, each storing the initial value `c`, `size()` is equal to `n`, and `capacity()` is a value at least as large as `size()`.

[-?- Remarks: Shall not participate in overload resolution if `Allocator` is a type that does not qualify as an allocator (26.2.1 [\[container.requirements.general\]](#)). [Note: This affects class template argument deduction. — end note]

[3079^{\(i\)}](#). LWG 2935 forgot to fix the existing_p overloads of `create_directory`

Section: 30.11.14.7 [\[fs.op.create_directory\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2018-03-07 **Last modified:** 2018-03-31

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2935](#) clarified that `create_directory` is not supposed to report an error if `exists(p)`, even if `p` is not a directory. However, the P/R there missed the existing_p overloads.

[2018-03-27 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 30.11.14.7 [[fs.op.create_directory](#)] as indicated:

```
bool create_directory(const path& p, const path& existing_p);
bool create_directory(const path& p, const path& existing_p, error_code& ec) noexcept;
```

-4- *Effects:* ~~Establishes the postcondition by attempting to create~~**Creates** the directory p resolves to, with attributes copied from directory existing_p. The set of attributes copied is operating system dependent. Creation failure because p ~~resolves to an existing directory shall not be treated as~~**already exists is not** an error. [Note: For POSIX-based operating systems, the attributes are those copied by native API `stat(existing_p.c_str(), &attributes_stat)` followed by `mkdir(p.c_str(), attributes_stat.st_mode)`. For Windows-based operating systems, the attributes are those copied by native API `CreateDirectoryExW(existing_p.c_str(), p.c_str(), 0)`. — end note]

~~-5- Postconditions: is_directory(p).~~

[...]

[3080^{\(i\)}](#). Floating point from_chars pattern specification breaks round-tripping

Section: 23.20.3 [[charconv.from.chars](#)] **Status:** [Tentatively Ready](#) **Submitter:** Greg Falcon **Opened:** 2018-03-12 **Last modified:** 2018-03-31

Priority: 0

View other [active issues](#) in [[charconv.from.chars](#)].

View all other [issues](#) in [[charconv.from.chars](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

from_chars specifies that the '+' character is never matched, but to_chars specifies its output format in terms of `printf()`, which puts a '+' sign before positive exponents.

Since `strtod()` matches '+' signs, it is also desirable to accept '+' in exponents, so that code currently using `strtod()` can be migrated to `from_chars()` without a breaking semantic change.

[2018-03-27 Moved to Tentatively Ready after 9 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 23.20.3 [[charconv.from.chars](#)] as indicated:

```
from_chars_result from_chars(const char* first, const char* last, float& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, double& value,
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, long double& value,
                             chars_format fmt = chars_format::general);
```

-6- *Requires:* fmt has the value of one of the enumerators of chars_format.

-7- *Effects:* The pattern is the expected form of the subject sequence in the "C" locale, as described for `strtod`, except that

(7.1) — the **only** sign **'+'** that may **only** appear **is '-'** in the exponent part;

(7.2) [...]

(7.3) [...]

(7.4) [...]

[3083^{\(i\)}](#). What should `ios::iword(-1)` do?

Section: 30.5.3.5 [\[ios.base.storage\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2018-03-16 **Last modified:** 2018-03-31

Priority: 0

View other [active issues](#) in `[ios.base.storage]`.

View all other [issues](#) in `[ios.base.storage]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

Is calling `iword` and `pword` with a negative argument undefined, or should it cause a failure condition (and return a valid reference)? What about `INT_MAX`? What about 0?

Using arbitrary indices isn't safe, because the implementation could be already using them for something else. Some replies on the reflector suggested the only reliable argument is one obtained from `ios_base::xalloc()`. Others pointed out that the `iwords` and `pwords` could be stored in sparse arrays, so that any value from `INT_MIN` to `INT_MAX` could be a valid key (which might require the implementation to use keys outside that range for its own entries in the arrays).

If it's undefined we should add a *Requires* element to the spec. If invalid indices are supposed to cause a failure we need to define which indices are invalid (and ensure that's something the implementation can check), and specify that it causes a failure.

[2018-03-27 Moved to Tentatively Ready after 5 positive votes on `c++std-lib`.]

Proposed resolution:

This wording is relative to [N4727](#).

1. Edit 30.5.3.5 [\[ios.base.storage\]](#) as indicated:

```
long& iword(int idx);
```

-?- Requires: `idx` is a value obtained by a call to `xalloc`.

-3- Effects: If `iarray` is a null pointer, [...]

[...]

```
void*& pword(int idx);
```

-?- Requires: `idx` is a value obtained by a call to `xalloc`.

-5- Effects: If `iarray` is a null pointer, [...]

[...]

[3094^{\(i\)}](#). §[time.duration.io]p4 makes surprising claims about encoding

Section: 23.17.5.10 [\[time.duration.io\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2018-04-02 **Last modified:** 2018-05-06

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

[time.duration.io]p4 says:

For streams where `charT` has an 8-bit representation, "`µs`" should be encoded as UTF-8. Otherwise UTF-16 or UTF-32 is encouraged. The implementation may substitute other encodings, including "`us`".

This choice of encoding is not up to the `<chrono>` library to decide or encourage. The basic execution character set determines how a `mu` should be encoded in type `char`, for instance, and it would be truly bizarre to use a UTF-8 encoding if that character set is, say, Latin-1 or EBCDIC.

I suggest we strike at least the first two sentences of this paragraph, as the meaning of the prior wording is unambiguous without them and confusing with them, and they do not providing any normative requirements (although they do provide recommendations). The third sentence appears to have a normative impact, but it's hard to see how it's legitimate to call "us" an "encoding" of "μs"; it's really just an alternative unit suffix. So how about replacing that paragraph with this:

If `Period::type` is `micro`, but the character `U+00B5` cannot be represented in the encoding used for `charT`, the unit suffix "us" is used instead of "μs".

(This also removes the permission for an implementation to choose an arbitrary alternative "encoding", which seems undesirable.)

[2018-04-23 Moved to Tentatively Ready after 6 positive votes on `c++std-lib`.]

Proposed resolution:

This wording is relative to [N4741](#).

1. Edit 23.17.5.10 [\[time.duration.io\]](#) as indicated:

```
template<class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

-1- *Requires*: [...]

-2- *Effects*: [...]

-3- The units suffix depends on the type `Period::type` as follows:

[...]

(3.5) — Otherwise, if `Period::type` is `micro`, the suffix is "μs" ("`\u00b5\u0073`").

[...]

(3.21) — Otherwise, the suffix is "`[num/den]`s".

[...]

-4- For streams where `charT` has an 8-bit representation, "μs" should be encoded as UTF-8. Otherwise UTF-16 or UTF-32 is encouraged. The implementation may substitute other encodings, including "us". If `Period::type` is `micro`, but the character `U+00B5` cannot be represented in the encoding used for `charT`, the unit suffix "us" is used instead of "μs".

-5- *Returns*: `os`.

[3100^{\(i\)}](#). Unnecessary and confusing "empty span" wording

Section: 26.7.3.2 [\[span.cons\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2018-04-12 **Last modified:** 2018-05-06

Priority: 0

View other [active issues](#) in [\[span.cons\]](#).

View all other [issues](#) in [\[span.cons\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

The `span` constructors have wording relics that mention an "empty span". It's unnecessary (the behavior is fully specified by the postconditions), but I left it there because I thought it was harmless. It was later pointed out to me that this is actually confusing. Talking about an "empty span" implies that there's just one such thing, but `span` permits `empty()` to be true while `data()` can vary (being null or non-null). (This behavior is very useful; consider how `equal_range()` behaves.)

To avoid confusion, the "empty span" wording should simply be removed, leaving the constructor behavior unchanged. Editorially, there's also a missing paragraph number.

[2018-04-24 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4741](#).

1. Edit 26.7.3.2 [\[span.cons\]](#) as indicated:

```
constexpr span() noexcept;
```

~~-1- Effects: Constructs an empty span.~~

-2- Postconditions: `size() == 0` && `data() == nullptr`.

-3- Remarks: This constructor shall not participate in overload resolution unless `Extent <= 0` is true.

```
constexpr span(pointer ptr, index_type count);
```

-4- Requires: `[ptr, ptr + count)` shall be a valid range. If extent is not equal to `dynamic_extent`, then `count` shall be equal to extent.

~~-5- Effects: Constructs a span that is a view over the range `[ptr, ptr + count)`. If `count` is 0 then an empty span is constructed.~~

-6- Postconditions: `size() == count` && `data() == ptr`.

~~-7- Throws: Nothing.~~

```
constexpr span(pointer first, pointer last);
```

-7- Requires: `[first, last)` shall be a valid range. If extent is not equal to `dynamic_extent`, then `last - first` shall be equal to extent.

~~-8- Effects: Constructs a span that is a view over the range `[first, last)`. If `last - first == 0` then an empty span is constructed.~~

-9- Postconditions: `size() == last - first` && `data() == first`.

-10- Throws: Nothing.

[3102^{\(i\)}](#). Clarify span iterator and const_iterator behavior

Section: 26.7.3.1 [\[span.overview\]](#) Status: [Tentatively Ready](#) Submitter: Stephan T. Lavavej Opened: 2018-04-12 Last modified: 2018-05-06

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

There are multiple issues with how span specifies its iterators:

- `const_iterator` isn't mentioned.
- The relationship between `iterator` and `const_iterator` isn't specified. (span isn't a container, so it doesn't receive this automatically.)
- The iterators should be specified to be `constexpr`.

By imitating 24.4.2.2 [\[string.view.iterators\]](#)/3 "All requirements on container iterators (`[container.requirements]`) apply to `basic_string_view::const_iterator` as well.", we can specify that iterator is convertible to `const_iterator`.

[2018-04-23 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4741](#).

1. Edit 26.7.3.1 [\[span.overview\]](#) as indicated:

-4- The iterator types `span` is a random access iterator and a contiguous iterator `span::iterator` and `span::const_iterator` are random access iterators (27.2.7 [\[random.access.iterators\]](#)), contiguous iterators (27.2.1 [\[iterator.requirements.general\]](#)), and constexpr iterators (27.2.1 [\[iterator.requirements.general\]](#)). All requirements on container iterators (26.2 [\[container.requirements\]](#)) apply to `span::iterator` and `span::const_iterator` as well.

3104⁽ⁱ⁾. Fixing duration division

Section: 23.17.5.5 [\[time.duration.nonmember\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Johel Ernesto Guerrero Peña **Opened:** 2018-04-17
Last modified: 2018-05-06

Priority: 0

View all other [issues](#) in `[time.duration.nonmember]`.

View all issues with [Tentatively Ready](#) status.

Discussion:

`[time.duration.nonmember]/1` states

In the function descriptions that follow, `CD` represents the return type of the function.

From what I could find, many definitions of `CD` in the paragraphs of `[time.duration.nonmember]` were lifted to `[time.duration.nonmember]/1` as cited above. That works for all other paragraphs, but not for `[time.duration.nonmember]/10`, which the change rendered ill-formed:

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr common_type_t<Rep1, Rep2>
operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

Returns: `CD(lhs).count() / CD(rhs).count()`.

In this case, we want `CD` to mean `common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>`. That way, the division has the expected semantics of dividing two quantities of the same dimension.

[2018-04-24 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4741](#).

1. Edit 23.17.5.5 [\[time.duration.nonmember\]](#) as indicated:

-1- In the function descriptions that follow, unless stated otherwise, let `CD` represents the return type of the function.

[...]

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr common_type_t<Rep1, Rep2>
operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

Let `CD` be `common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>`.

-10- *Returns:* `CD(lhs).count() / CD(rhs).count()`.